Tesseract: Technical Build Specification for Polkadot Solidity Hackathon

Document Version: 1.0
Target Hackathon: Polkadot Solidity Hackathon (Feb 15 - Mar 24, 2026)
Build Duration: 18 days (Mar 2 - Mar 20, 2026)
Target Tracks: Track 1 (EVM Smart Contracts) + Track 2 (PVM Smart Contracts)

Executive Summary

Tesseract is a modular cross-chain DeFi primitive hub consisting of three interconnected smart contracts deployed on Polkadot Hub. It provides developers with composable building blocks for cross-chain liquidity routing, native DOT liquid staking, and automated yield strategies—functionality that cannot be replicated on Ethereum alone.

Core Value Proposition:

- First DeFi protocol to fully exploit Polkadot Hub's XCM precompile for seamless cross-chain operations[1]
- Native integration with Polkadot's staking system via precompiles (impossible on Ethereum)
- Single-transaction cross-chain strategies across multiple parachains
- Open-source, modular architecture for ecosystem-wide composability

Hackathon Alignment:

- **Track 1 (EVM):** TesseractSwap AMM with XC-20 asset support

- **Track 2 (PVM):** TesseractStake with native staking precompile integration
- **Prize Categories:** DeFi-enabled dapps, applications using Polkadot native assets, accessing Polkadot native functionality via precompiles

---

Architecture Overview

System Components

Figure 1: Tesseract Architecture - Three-Module System

Module	Track	Technology	Primary Function
TesseractSwap	Track 1	Solidity (EVM)	XC-20 asset AMM DEX
TesseractStake	Track 2	Solidity (PVM)	Native DOT liquid staking
TesseractVault	Both	Solidity (EVM/PVM)	Cross-chain yield routing

Table 1: Module breakdown and hackathon track alignment

Technology Stack

- **Smart Contract Language:** Solidity 0.8.20+
- **Development Framework:** Foundry (primary), Hardhat (secondary for scripts)
- **Testing:** Foundry tests + Chopsticks for XCM simulation
- **Frontend:** Next.js 14, Wagmi v2, Viem, TailwindCSS
- **RPC Providers:** Polkadot Hub Testnet (Paseo/Westend)
- **Deployment Target:** Polkadot Hub Testnet initially, Polkadot Hub Mainnet post-hackathon

---

Module 1: TesseractSwap (Track 1 - EVM Smart Contracts)

Purpose

An automated market maker (AMM) decentralized exchange that natively supports XC-20 cross-chain assets from Polkadot parachains. Unlike standard Uniswap V2 forks, TesseractSwap integrates the XCM precompile to enable cross-chain asset transfers as part of swap operations[2].

Technical Specifications

Contract Name: TesseractSwap.sol

Core Functionality:

1. Constant product AMM (x * y = k formula from Uniswap V2)
2. Liquidity pool creation for any ERC-20 or XC-20 token pair
3. Add/remove liquidity with LP token minting
4. Token swaps with 0.3% fee to liquidity providers
5. XCM-enabled cross-chain asset deposits via precompile integration

Key Interfaces:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

interface ITesseractSwap {
// Core AMM functions
function createPair(address tokenA, address tokenB) external returns
(address pair);
function addLiquidity(
address tokenA,
address tokenB,
uint256 amountA,
uint256 amountB,
uint256 minAmountA,
uint256 minAmountB,
address to,
```

uint256 deadline
) external returns (uint256 liquidity);

```
function removeLiquidity(
    address tokenA,
    address tokenB,
    uint256 liquidity,
    uint256 minAmountA,
    uint256 minAmountB,
    address to,
    uint256 deadline
) external returns (uint256 amountA, uint256 amountB);

function swap(
    address tokenIn,
    address tokenOut,
    uint256 amountIn,
    uint256 minAmountOut,
    address to,
    uint256 deadline
) external returns (uint256 amountOut);

// XCM-enhanced functions
function swapAndBridgeToParachain(
    address tokenIn,
    address tokenOut,
    uint256 amountIn,
    bytes calldata destination,
    bytes calldata xcmMessage
) external returns (uint256 amountOut);
```

}

XC-20 Integration Pattern:

XC-20 tokens are Substrate-native assets that appear as ERC-20 tokens in the EVM environment via precompiles[3]. Each XC-20 has a deterministic precompile address calculated as:

```
// XC-20 assets use precompile addresses
// Address format: 0xFFFFFFFF + asset_id (32 bits)
// Example: Asset ID 1 →
0xFFFFFFFF0000000000000000000000000000001

interface IERC20Precompile {
function balanceOf(address account) external view returns (uint256);
function transfer(address to, uint256 amount) external returns
(bool);
function approve(address spender, uint256 amount) external returns
(bool);
function transferFrom(address from, address to, uint256 amount)
external returns (bool);
}
```

XCM Precompile Integration:

```
// XCM Precompile Interface (fixed address)
address constant XCM_PRECOMPILE =
0x0000000000000000000000000000000000000a0000;

interface IXcm {
struct Weight {
uint64 refTime;
uint64 proofSize;
}

function send(bytes calldata destination, bytes calldata message) external;
function execute(bytes calldata message, Weight calldata weight) external;
function weighMessage(bytes calldata message) external view returns (Weig

}
```

Cross-Chain Swap Flow:

1. User initiates swap on Polkadot Hub
2. TesseractSwap executes local swap (tokenIn → tokenOut)
3. Contract calls XCM precompile to send tokenOut to destination parachain
4. XCM message includes beneficiary account on destination chain

5. User receives swapped tokens on target parachain

Implementation Details

File Structure:

```
contracts/
├── TesseractSwap.sol # Main AMM contract
├── TesseractPair.sol # Individual liquidity pair
├── TesseractFactory.sol # Pair deployment factory
├── libraries/
│ ├── SwapLibrary.sol # AMM math utilities
│ └── XCMHelpers.sol # XCM message encoding helpers
└── interfaces/
├── ITesseractSwap.sol
└── IXCM.sol # XCM precompile interface
```

Critical Implementation Notes:

1. **Reentrancy Protection:** Use OpenZeppelin's ReentrancyGuard for all state-changing functions
2. **Overflow Protection:** Solidity 0.8.x has built-in overflow checks; no SafeMath needed
3. **Deadline Checks:** All swap functions must validate `block.timestamp <= deadline`
4. **Minimum Output:** Implement slippage protection via `minAmountOut` parameters
5. **Fee Calculation:** 0.3% fee: `amountWithFee = amountIn * 997 / 1000`

Testing Requirements:

- Unit tests for AMM math (constant product formula correctness)
- Integration tests with mock XC-20 tokens
- Fork tests against Polkadot Hub Testnet
- XCM message encoding/decoding validation
- Gas benchmarking for swap operations

Code Reference - Base AMM Implementation

Starting point (modify to add XCM layer):

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import "@openzeppelin/contracts/security/ReentrancyGuard.sol";

contract TesseractPair is ERC20, ReentrancyGuard {
address public token0;
address public token1;
```

```
uint256 private reserve0;
uint256 private reserve1;

uint256 private constant MINIMUM_LIQUIDITY = 10**3;

constructor(address _token0, address _token1) ERC20("Tesseract LP", "TLP") {
    token0 = _token0;
    token1 = _token1;
}

function getReserves() public view returns (uint256 _reserve0, uint256 _rese
    _reserve0 = reserve0;
    _reserve1 = reserve1;
}

function mint(address to) external nonReentrant returns (uint256 liquidity)
    (uint256 _reserve0, uint256 _reserve1) = getReserves();
    uint256 balance0 = IERC20(token0).balanceOf(address(this));
    uint256 balance1 = IERC20(token1).balanceOf(address(this));
    uint256 amount0 = balance0 - _reserve0;
    uint256 amount1 = balance1 - _reserve1;

    uint256 _totalSupply = totalSupply();
    if (_totalSupply == 0) {
        liquidity = sqrt(amount0 * amount1) - MINIMUM_LIQUIDITY;
```

```
        _mint(address(0), MINIMUM_LIQUIDITY); // Lock initial liquidity
    } else {
        liquidity = min((amount0 * _totalSupply) / _reserve0, (amount1 * _totalSu
    }

    require(liquidity > 0, "Insufficient liquidity minted");
    _mint(to, liquidity);

    _update(balance0, balance1);
}

function swap(uint256 amount0Out, uint256 amount1Out, address to) exterr
    require(amount0Out > 0 || amount1Out > 0, "Insufficient output amount")
    (uint256 _reserve0, uint256 _reserve1) = getReserves();
    require(amount0Out < _reserve0 && amount1Out < _reserve1, "Insufficien

    if (amount0Out > 0) IERC20(token0).transfer(to, amount0Out);
    if (amount1Out > 0) IERC20(token1).transfer(to, amount1Out);

    uint256 balance0 = IERC20(token0).balanceOf(address(this));
    uint256 balance1 = IERC20(token1).balanceOf(address(this));

    uint256 amount0In = balance0 > _reserve0 - amount0Out ? balance0 - (_res
    uint256 amount1In = balance1 > _reserve1 - amount1Out ? balance1 - (_res
    require(amount0In > 0 || amount1In > 0, "Insufficient input amount");

    // Verify constant product formula with 0.3% fee
    uint256 balance0Adjusted = (balance0 * 1000) - (amount0In * 3);
    uint256 balance1Adjusted = (balance1 * 1000) - (amount1In * 3);
    require(balance0Adjusted * balance1Adjusted >= _reserve0 * _reserve1 * (

    _update(balance0, balance1);
}

function _update(uint256 balance0, uint256 balance1) private {
    reserve0 = balance0;
    reserve1 = balance1;
}
```

```
function sqrt(uint256 y) internal pure returns (uint256 z) {
    if (y > 3) {
        z = y;
        uint256 x = y / 2 + 1;
        while (x < z) {
            z = x;
            x = (y / x + x) / 2;
        }
    } else if (y != 0) {
        z = 1;
    }
}

function min(uint256 a, uint256 b) internal pure returns (uint256) {
    return a < b ? a : b;
}
```

}

XCM Layer Addition:

Add this functionality to enable cross-chain operations:

// Add to TesseractSwap.sol
function swapAndBridgeXCM(
address tokenIn,
address tokenOut,
uint256 amountIn,
bytes calldata destinationParachain,
bytes calldata xcmMessage
) external returns (uint256 amountOut) {
// 1. Execute local swap
amountOut = _executeSwap(tokenIn, tokenOut, amountIn);

```
// 2. Call XCM precompile to bridge output tokens
IXcm xcm = IXcm(XCM_PRECOMPILE);
xcm.send(destinationParachain, xcmMessage);
```

```
emit CrossChainSwap(msg.sender, tokenIn, tokenOut, amountIn, amountOu
```

}

Module 2: TesseractStake (Track 2 - PVM Smart Contracts)

Purpose

A liquid staking protocol that interacts directly with Polkadot's native staking system via runtime precompiles. Users stake DOT and receive `lstDOT` (Liquid Staked DOT) tokens that represent their staking position while remaining transferable and usable in DeFi[4].

Technical Specifications

Contract Name: `TesseractStake.sol`

Core Functionality:

1. Accept DOT deposits and stake via native staking precompile
2. Mint lstDOT tokens (ERC-20) representing staked position
3. Unstake DOT and burn lstDOT tokens
4. Distribute staking rewards to lstDOT holders
5. Integration with PVM for performance-critical reward calculations

Key Interfaces:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

interface ITesseractStake {
function stake(uint256 amount) external returns (uint256
lstDOTAmount);
function unstake(uint256 lstDOTAmount) external returns (uint256
dotAmount);
function claimRewards() external returns (uint256 rewards);
function exchangeRate() external view returns (uint256);
```

function totalStaked() external view returns (uint256);
function totalLstDOT() external view returns (uint256);
}

Native Staking Integration:

Polkadot Hub does not yet have a documented public staking precompile address (as of March 2026). Based on common precompile patterns and the System precompile documentation[5], we use a placeholder address and interface that follows the expected pattern:

// NOTE: This is a template interface based on common staking precompile patterns
// Actual address and interface should be verified from official Polkadot Hub documentation
// Expected pattern similar to other chains:
0x0000000000000000000000000000000000000800

address constant STAKING_PRECOMPILE =
0x0000000000000000000000000000000000000800;

interface IStakingPrecompile {
// Stake DOT with the contract acting as nominator
function stake(uint256 amount) external returns (bool);

```
// Unstake DOT (initiates unbonding period)
function unstake(uint256 amount) external returns (bool);

// Withdraw unbonded DOT after unbonding period
function withdraw() external returns (uint256);

// Get current staking rewards
function getRewards(address account) external view returns (uint256);

// Nominate validators
function nominate(bytes32[] calldata validators) external returns (bool);
```

}

Important Note on Staking Precompile:

As of March 2026, Polkadot Hub's staking precompile interface is still being finalized. For hackathon purposes:

- Use mock staking precompile for local testing
- Monitor Polkadot Hub documentation for official precompile release: https://docs.polkadot.com/smart-contracts/precompiles/
- Check OpenGuild Discord and hackathon channels for updates
- If precompile not available by mid-March, implement simplified version that demonstrates the architecture pattern

Alternative Implementation Path:

If native staking precompile is unavailable during hackathon:

1. Deploy TesseractStake as EVM contract on Polkadot Hub
2. Implement *simulated* staking that accepts DOT deposits
3. Use XCM to send DOT to Bifrost parachain for liquid staking (integrate with existing vDOT)[6]
4. Bridge vDOT back to Hub and wrap as lstDOT
5. Document this as "cross-chain liquid staking aggregator" approach

This alternative still qualifies for Track 2 categories: "Applications using Polkadot native Assets" and demonstrates XCM integration.

Implementation Details

File Structure:

```
contracts/
├── TesseractStake.sol # Main liquid staking contract
├── LstDOT.sol # ERC-20 liquid staking token
├── libraries/
│ ├── StakingMath.sol # Reward distribution calculations (PVM-optimized)
│ └── ValidatorSelector.sol # Validator nomination logic
└── interfaces/
├── ITesseractStake.sol
└── IStakingPrecompile.sol # Native staking interface
```

Core Logic - Exchange Rate Mechanism:

The exchange rate between DOT and lstDOT increases over time as staking rewards accumulate:

function exchangeRate() public view returns (uint256) {
uint256 totalLst = totalSupply(); // Total lstDOT supply
if (totalLst == 0) return 1e18; // Initial rate: 1 DOT = 1 lstDOT

```
uint256 totalValue = totalStakedDOT + accumulatedRewards;
return (totalValue * 1e18) / totalLst; // Rate in 18 decimals
```

}

function stake(uint256 dotAmount) external returns (uint256 lstDOTAmount) {
require(dotAmount > 0, "Amount must be positive");

```
// Transfer DOT from user
require(DOT_TOKEN.transferFrom(msg.sender, address(this), dotAmount), "

// Stake via precompile
IStakingPrecompile(STAKING_PRECOMPILE).stake(dotAmount);

// Calculate lstDOT to mint based on current exchange rate
uint256 rate = exchangeRate();
lstDOTAmount = (dotAmount * 1e18) / rate;

// Mint lstDOT to user
_mint(msg.sender, lstDOTAmount);

totalStakedDOT += dotAmount;
emit Staked(msg.sender, dotAmount, lstDOTAmount);
```

}

Reward Distribution:

Staking rewards automatically increase the exchange rate, so lstDOT holders automatically accrue value:

```
function updateRewards() external {
uint256 newRewards =
IStakingPrecompile(STAKING_PRECOMPILE).getRewards(address(thi
s));
accumulatedRewards += newRewards;
emit RewardsUpdated(newRewards, accumulatedRewards);
}
```

PVM Integration for Performance:

TesseractStake should use PVM (PolkaVM) for computationally expensive operations like reward distribution across many users. This demonstrates Track 2's "Call Rust or C++ libraries from Solidity" category.

PVM Pattern (Conceptual):

```
// Import PVM-compiled Rust library for efficient calculations
library StakingMathPVM {
// Call PVM-compiled Rust function via FFI
// This performs complex financial calculations faster than EVM
function calculateRewardDistribution(
uint256[] memory stakingAmounts,
uint256 totalRewards
) internal returns (uint256[] memory) {
// PVM FFI call to Rust implementation
// Returns array of individual reward amounts
}
}
```

Note: PVM FFI integration is advanced. For hackathon MVP, implement calculations in pure Solidity first, document where PVM optimization would apply, and add PVM integration if time permits.

Testing Requirements

- Unit tests for exchange rate calculations
- Mock staking precompile for isolated testing
- Integration tests with Polkadot Hub Testnet (when precompile available)
- Stress testing with multiple stakers and reward cycles

- Gas benchmarking vs EVM-only implementation

Module 3: TesseractVault (Cross-Chain Yield Strategies)

Purpose

An automated yield optimizer that deploys lstDOT into the highest-yielding strategies across Polkadot parachains. Users deposit lstDOT, and the vault automatically routes funds to Bifrost, Acala, or other DeFi parachains via XCM, optimizing for best APY[7].

Technical Specifications

Contract Name: `TesseractVault.sol`

Core Functionality:

1. Accept lstDOT deposits from users
2. Analyze yields across multiple parachains
3. Execute cross-chain deposits via XCM
4. Harvest rewards and compound automatically
5. Withdraw and return funds to users on demand

Key Interfaces:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

interface ITesseractVault {
function deposit(uint256 lstDOTAmount) external returns (uint256 shares);
function withdraw(uint256 shares) external returns (uint256 lstDOTAmount);
function harvest() external returns (uint256 rewards);
function rebalance() external;
function getAPY() external view returns (uint256);
function totalAssets() external view returns (uint256);
}
```

Strategy Structure:

```
struct Strategy {
bytes32 parachainId; // Target parachain identifier
address protocolAddress; // Protocol contract on destination chain
uint256 allocatedAmount; // Amount currently allocated
uint256 targetAllocation; // Target allocation percentage (basis points)
uint256 lastAPY; // Last recorded APY
bool active; // Strategy active status
}
```

Cross-Chain Yield Flow:

1. Vault receives lstDOT deposits on Polkadot Hub
2. Vault analyzes current APY across integrated protocols:
    - Bifrost SLP (vDOT liquid staking)
    - Acala aUSD stablecoin pools
    - Moonbeam Stellaswap liquidity mining
3. Vault rebalances allocations to maximize yield
4. XCM messages sent to deploy assets to target parachains
5. Rewards harvested and compounded automatically
6. Users can withdraw proportional share at any time

Implementation Details

File Structure:

```
contracts/
├── TesseractVault.sol # Main vault contract (ERC-4626 compatible)
├── strategies/
│   ├── BifrostStrategy.sol # Bifrost SLP integration
│   ├── AcalaStrategy.sol # Acala lending integration
│   └── BaseStrategy.sol # Abstract strategy base
├── libraries/
│   ├── YieldAggregator.sol # APY calculation and comparison
│   └── XCMRouter.sol # Cross-chain message routing
└── interfaces/
    ├── ITesseractVault.sol
    ├── IStrategy.sol
    └── IXCM.sol
```

ERC-4626 Vault Standard:

Implement ERC-4626 tokenized vault standard for maximum composability:

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import
"@openzeppelin/contracts/token/ERC20/extensions/ERC4626.sol";

contract TesseractVault is ERC4626 {
IERC20 public lstDOT;
IXcm public xcmPrecompile;

```
mapping(bytes32 => Strategy) public strategies;
bytes32[] public activeStrategies;

constructor(
    address _lstDOT,
    address _xcmPrecompile
) ERC4626(IERC20(_lstDOT)) ERC20("Tesseract Vault", "tvLSTDOT") {
    lstDOT = IERC20(_lstDOT);
    xcmPrecompile = IXcm(_xcmPrecompile);
}

function totalAssets() public view override returns (uint256) {
    uint256 total = lstDOT.balanceOf(address(this)); // Idle balance

    // Add allocated amounts across all strategies
    for (uint256 i = 0; i < activeStrategies.length; i++) {
        total += strategies[activeStrategies[i]].allocatedAmount;
    }

    return total;
}

function rebalance() external {
    // 1. Calculate optimal allocation based on APYs
    uint256[] memory optimalAllocations = _calculateOptimalAllocation();
```

```
    // 2. Execute rebalancing via XCM
    for (uint256 i = 0; i < activeStrategies.length; i++) {
        bytes32 strategyId = activeStrategies[i];
        Strategy storage strategy = strategies[strategyId];

        uint256 currentAlloc = strategy.allocatedAmount;
        uint256 targetAlloc = optimalAllocations[i];

        if (targetAlloc > currentAlloc) {
            // Increase allocation - send more funds
            _depositToStrategy(strategyId, targetAlloc - currentAlloc);
        } else if (targetAlloc < currentAlloc) {
            // Decrease allocation - withdraw funds
            _withdrawFromStrategy(strategyId, currentAlloc - targetAlloc);
        }
    }
}

function _depositToStrategy(bytes32 strategyId, uint256 amount) internal {
    Strategy storage strategy = strategies[strategyId];

    // Encode XCM message for cross-chain deposit
    bytes memory xcmMessage = _encodeDepositXCM(
        strategy.parachainId,
        strategy.protocolAddress,
        amount
    );

    // Send via XCM precompile
    xcmPrecompile.send(
        abi.encode(strategy.parachainId),
        xcmMessage
    );

    strategy.allocatedAmount += amount;
}

function _encodeDepositXCM(
```

```
    bytes32 parachainId,
    address protocol,
    uint256 amount
) internal pure returns (bytes memory) {
    // Simplified XCM message encoding
    // Real implementation would use proper SCALE encoding
    return abi.encodePacked(
        parachainId,
        protocol,
        amount
    );
}
```

}

APY Aggregation Logic:

library YieldAggregator {
struct YieldData {
bytes32 strategyId;
uint256 apy;
uint256 tvl;
uint256 capacity;
}

```
function calculateOptimalAllocation(
    YieldData[] memory yields,
    uint256 totalToAllocate
) internal pure returns (uint256[] memory allocations) {
    allocations = new uint256[](yields.length);

    // Sort by APY descending
    _sortByAPY(yields);

    // Allocate to highest APY strategies up to capacity
    uint256 remaining = totalToAllocate;
    for (uint256 i = 0; i < yields.length && remaining > 0; i++) {
        uint256 allocation = min(remaining, yields[i].capacity);
```

```
        allocations[i] = allocation;
        remaining -= allocation;
    }

    return allocations;
}

function min(uint256 a, uint256 b) internal pure returns (uint256) {
    return a < b ? a : b;
}

function _sortByAPY(YieldData[] memory data) internal pure {
    // Simple bubble sort (optimize for production)
    for (uint256 i = 0; i < data.length; i++) {
        for (uint256 j = i + 1; j < data.length; j++) {
            if (data[j].apy > data[i].apy) {
                YieldData memory temp = data[i];
                data[i] = data[j];
                data[j] = temp;
            }
        }
    }
}
```

}

XCM Message Patterns for Strategies

Deposit to Bifrost SLP:

function encodeBifrostDeposit(uint256 amount) internal pure
returns (bytes memory) {
// XCM message structure for Bifrost liquid staking
// WithdrawAsset + BuyExecution + Transact (call mint_vdot)
return abi.encodePacked(
hex"030c", // WithdrawAsset instruction
_encodeMultiAsset(amount),
hex"0401", // BuyExecution
hex"0d01", // Transact

```
_encodeBifrostMintCall(amount)
);
}
```

Withdraw from Acala:

```
function encodeAcalaWithdraw(uint256 shares) internal pure
returns (bytes memory) {
// XCM message to redeem from Acala lending
return abi.encodePacked(
hex"030c",
_encodeMultiAsset(shares),
hex"0401",
hex"0d01",
_encodeAcalaRedeemCall(shares)
);
}
```

Note: These are simplified representations. Production XCM messages require proper SCALE encoding using libraries like `@polkadot/api` or custom Rust helpers.

Testing Requirements

- Unit tests for vault accounting and share calculations
- Integration tests with mock XCM responses
- Simulated multi-strategy rebalancing scenarios
- Performance testing for gas costs of rebalancing
- End-to-end test with Chopsticks for XCM simulation

Development Environment Setup

Prerequisites

- Node.js 18+ and npm/yarn
- Foundry (forge, cast, anvil)
- Polkadot.js browser extension
- MetaMask (configured for Polkadot Hub)
- Git and GitHub account

Installation Steps

1. Install Foundry:

```
curl -L https://foundry.paradigm.xyz | bash
foundryup
```

2. Initialize Project:

```
mkdir tesseract-defi
cd tesseract-defi
forge init
```

3. Install Dependencies:

```
forge install OpenZeppelin/openzeppelin-contracts
forge install foundry-rs/forge-std
```

4. Configure Foundry:

Create `foundry.toml`:

```
[profile.default]
src = "src"
out = "out"
libs = ["lib"]
solc_version = "0.8.20"
optimizer = true
optimizer_runs = 200

[rpc_endpoints]
polkadot_hub_testnet = "https://polkadot-hub-testnet.example.com"
```

Update with actual Polkadot Hub Testnet RPC

```
[etherscan]
polkadot_hub = { key = "${POLKADOT_HUB_API_KEY}" }
```

5. Set Up Environment Variables:

Create .env:

```
PRIVATE_KEY=your_private_key_here
POLKADOT_HUB_RPC=https://polkadot-hub-testnet-rpc-url
ETHERSCAN_API_KEY=your_api_key_if_available
```

Connecting to Polkadot Hub Testnet

Network Configuration:

Parameter	Value
Network Name	Polkadot Hub Testnet
RPC URL	wss://westend-hub-rpc.polkadot.io
Chain ID	TBD (check docs)
Currency Symbol	WND (Westend)
Block Explorer	https://polkadot.subscan.io

Table 2: Polkadot Hub Testnet network parameters

Add Network to MetaMask:

1. Open MetaMask
2. Click network dropdown → "Add Network"
3. Enter Polkadot Hub Testnet parameters
4. Save and switch to network

Get Test Tokens:

Visit Westend Faucet: https://faucet.polkadot.io/westend[8]

1. Enter your Polkadot address
2. Request WND tokens
3. Wait 1-2 minutes for confirmation

Local Development Node (Optional)

For faster iteration without testnet latency:

Clone Polkadot SDK

git clone https://github.com/paritytech/polkadot-sdk.git
cd polkadot-sdk

Build local node

cargo build --release

Run local dev node with instant finality

./target/release/polkadot --dev --tmp

Connect to local node:

- RPC: http://localhost:9933
- WS: ws://localhost:9944

Smart Contract Development Workflow

Phase 1: TesseractSwap (Days 1-6)

Day 1-2: Core AMM Logic

1. Implement TesseractPair.sol with constant product formula
2. Add liquidity provision functions (addLiquidity, removeLiquidity)
3. Implement swap function with 0.3% fee
4. Write comprehensive unit tests

Day 3-4: XC-20 Integration

1. Create mock XC-20 tokens for testing
2. Add XC-20 detection logic to pair creation
3. Implement cross-chain asset deposit via XCM precompile
4. Test XCM message encoding/decoding

Day 5-6: Factory and Router

1. Implement TesseractFactory for pair deployment
2. Create router contract for multi-hop swaps
3. Add front-running protection (deadline, slippage)
4. Deploy to Polkadot Hub Testnet

Deployment Command:

```
forge create src/TesseractSwap.sol:TesseractSwap
--rpc-url $POLKADOT_HUB_RPC
--private-key $PRIVATE_KEY
--constructor-args $XCM_PRECOMPILE_ADDRESS
--verify
```

Phase 2: TesseractStake (Days 7-12)

Day 7-8: Staking Precompile Research

1. Check latest Polkadot Hub documentation for staking precompile
2. Join OpenGuild Discord for technical support
3. Create mock precompile interface if official not yet available
4. Design lstDOT token contract (ERC-20)

Day 9-10: Core Staking Logic

1. Implement stake() function with exchange rate calculation
2. Add unstake() with unbonding period handling
3. Implement reward distribution mechanism
4. Write unit tests with mocked staking precompile

Day 11-12: Integration and Testing

1. Deploy lstDOT token contract
2. Deploy TesseractStake contract
3. Test full stake → wait → unstake cycle
4. Verify reward accrual and exchange rate updates

Alternative Path (If Precompile Unavailable):

1. Implement cross-chain liquid staking via Bifrost
2. Use XCM to bridge DOT to Bifrost parachain
3. Receive vDOT, wrap as lstDOT on Hub

4. Document as "cross-chain liquid staking aggregator"

Phase 3: TesseractVault (Days 13-17)

Day 13-14: Vault Foundation

1. Implement ERC-4626 vault standard
2. Add deposit/withdraw with share accounting
3. Create Strategy base contract interface
4. Write vault accounting tests

Day 15-16: Strategy Integration

1. Implement BifrostStrategy (vDOT staking)
2. Implement AcalaStrategy (aUSD lending)
3. Add YieldAggregator library for APY comparison
4. Create rebalance() function with XCM routing

Day 17: End-to-End Testing

1. Deploy complete system to testnet
2. Test multi-strategy allocation
3. Simulate rebalancing scenarios
4. Verify cross-chain message execution

Phase 4: Frontend and Documentation (Days 18-20)

Day 18: Frontend Development

1. Next.js app with Wagmi/Viem integration
2. Wallet connection (MetaMask, Polkadot.js)
3. UI for TesseractSwap (swap interface)
4. UI for TesseractStake (stake/unstake)
5. UI for TesseractVault (deposit/withdraw, APY display)

Day 19: Documentation

1. Complete README.md with architecture diagrams
2. Add inline code documentation
3. Write deployment guide
4. Create demo video script

Day 20: Final Testing and Submission

1. End-to-end integration testing
2. Record 3-5 minute demo video
3. Prepare presentation slides
4. Submit to hackathon platform before March 20 deadline

Testing Strategy

Unit Testing with Foundry

Test Structure:

```
test/
├── TesseractSwap.t.sol # AMM logic tests
├── TesseractStake.t.sol # Staking mechanism tests
├── TesseractVault.t.sol # Vault accounting tests
├── mocks/
│  ├── MockXCM.sol # Mock XCM precompile
│  ├── MockStaking.sol # Mock staking precompile
│  └── MockXC20.sol # Mock XC-20 tokens
└── integration/
└── E2E.t.sol # Full workflow tests
```

Example Test - TesseractSwap:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "forge-std/Test.sol";
import "../src/TesseractSwap.sol";
import "./mocks/MockXC20.sol";

contract TesseractSwapTest is Test {
TesseractSwap public swap;
MockXC20 public tokenA;
MockXC20 public tokenB;

    address public alice = address(0x1);
    address public bob = address(0x2);

    function setUp() public {
```

```
    swap = new TesseractSwap();
    tokenA = new MockXC20("Token A", "TKA");
    tokenB = new MockXC20("Token B", "TKB");

    // Mint tokens to test accounts
    tokenA.mint(alice, 1000 ether);
    tokenB.mint(alice, 1000 ether);
}

function testCreatePairAndAddLiquidity() public {
    vm.startPrank(alice);

    // Approve tokens
    tokenA.approve(address(swap), 100 ether);
    tokenB.approve(address(swap), 100 ether);

    // Add initial liquidity
    (uint256 liquidity) = swap.addLiquidity(
        address(tokenA),
        address(tokenB),
        100 ether,
        100 ether,
        90 ether,  // Min amount A
        90 ether,  // Min amount B
        alice,
        block.timestamp + 3600
    );

    assertGt(liquidity, 0, "Should receive LP tokens");
    vm.stopPrank();
}

function testSwapExactTokensForTokens() public {
    // First add liquidity
    testCreatePairAndAddLiquidity();

    vm.startPrank(bob);
    tokenA.mint(bob, 10 ether);
```

```
    tokenA.approve(address(swap), 10 ether);

    uint256 balanceBefore = tokenB.balanceOf(bob);

    // Execute swap
    uint256 amountOut = swap.swap(
        address(tokenA),
        address(tokenB),
        10 ether,
        1 ether,  // Min amount out (slippage protection)
        bob,
        block.timestamp + 3600
    );

    uint256 balanceAfter = tokenB.balanceOf(bob);
    assertEq(balanceAfter - balanceBefore, amountOut, "Should receive correc
    vm.stopPrank();
}

function testSwapRevertsOnExpiredDeadline() public {
    testCreatePairAndAddLiquidity();

    vm.startPrank(bob);
    tokenA.mint(bob, 10 ether);
    tokenA.approve(address(swap), 10 ether);

    // Try swap with past deadline
    vm.expectRevert("Deadline expired");
    swap.swap(
        address(tokenA),
        address(tokenB),
        10 ether,
        1 ether,
        bob,
        block.timestamp - 1  // Deadline in past
    );
```

```
    vm.stopPrank();
  }
```

}

Run Tests:

forge test -vvv

Generate Coverage Report:

forge coverage --report lcov

Integration Testing with Chopsticks

Chopsticks is a tool for simulating Polkadot network state and testing XCM messages locally[2].

Install Chopsticks:

npm install -g @acala-network/chopsticks

Configuration:

Create `chopsticks.config.yml`:

relaychain:
endpoint: wss://westend-rpc.polkadot.io

parachains:

- endpoint: wss://westend-hub-rpc.polkadot.io
  paraId: 1000
- endpoint: wss://bifrost-westend-rpc.polkadot.io
  paraId: 2001

Run XCM Simulation:

chopsticks --config=chopsticks.config.yml

Test XCM Messages:

```
// test/integration/xcm-test.ts
import { ApiPromise } from '@polkadot/api';

describe('TesseractVault XCM Integration', () => {
it('should successfully deposit to Bifrost via XCM', async () => {
const api = await ApiPromise.create({
provider: 'ws://localhost:8000' // Chopsticks endpoint
});

  // Simulate XCM message from Hub to Bifrost
  const message = {
    WithdrawAsset: [...],
    BuyExecution: {...},
    Transact: {...}
  };

  // Execute and verify
  const result = await api.tx.xcmPallet.send(
    { V3: { parents: 1, interior: { X1: { Parachain: 2001 } } } },
    message
  ).signAndSend(signer);

  expect(result.isOk).toBe(true);

});
});
```

Frontend Development

Technology Stack

- **Framework:** Next.js 14 with App Router
- **Styling:** TailwindCSS + shadcn/ui components
- **Web3 Integration:** Wagmi v2 + Viem
- **State Management:** Zustand or Jotai
- **Charts:** Recharts or Chart.js

Project Structure

```
frontend/
├── app/
│   ├── layout.tsx # Root layout with providers
│   ├── page.tsx # Landing page
│   ├── swap/
│   │   └── page.tsx # TesseractSwap interface
│   ├── stake/
│   │   └── page.tsx # TesseractStake interface
│   └── vault/
│       └── page.tsx # TesseractVault interface
├── components/
│   ├── WalletConnect.tsx # Wallet connection component
│   ├── SwapWidget.tsx # Swap interface
│   ├── StakePanel.tsx # Staking interface
│   └── VaultDashboard.tsx # Vault stats and controls
├── hooks/
│   ├── useSwap.ts # Swap contract interactions
│   ├── useStake.ts # Stake contract interactions
│   └── useVault.ts # Vault contract interactions
├── lib/
│   ├── contracts.ts # Contract ABIs and addresses
│   └── wagmi.ts # Wagmi configuration
└── public/
    └── assets/ # Logo, images
```

Core Components

Wallet Connection:

```
// components/WalletConnect.tsx
'use client';

import { useAccount, useConnect, useDisconnect } from 'wagmi';
import { Button } from '@/components/ui/button';

export function WalletConnect() {
const { address, isConnected } = useAccount();
const { connect, connectors } = useConnect();
const { disconnect } = useDisconnect();
```

```
if (isConnected) {
return (
{address?.slice(0, 6)}...{address?.slice(-4)}
<Button onClick={() => disconnect()}>Disconnect</Button>

);
}

return (

{connectors.map((connector) => (
<Button
key={connector.id}
onClick={() => connect({ connector })}
>
Connect {connector.name}
</Button>
))}

);
}
```

Swap Widget:

```
// components/SwapWidget.tsx
'use client';

import { useState } from 'react';
import { useWriteContract, useReadContract } from 'wagmi';
import { parseEther, formatEther } from 'viem';
import { TESSERACT_SWAP_ABI, TESSERACT_SWAP_ADDRESS } from
'@/lib/contracts';

export function SwapWidget() {
const [tokenIn, setTokenIn] = useState('');
const [tokenOut, setTokenOut] = useState('');
const [amountIn, setAmountIn] = useState('');
const [amountOut, setAmountOut] = useState('0');

const { writeContract } = useWriteContract();
```

```
// Get quote for swap
const { data: quote } = useReadContract({
address: TESSERACT_SWAP_ADDRESS,
abi: TESSERACT_SWAP_ABI,
functionName: 'getAmountOut',
args: [parseEther(amountIn || '0'), tokenIn, tokenOut],
});

const handleSwap = async () => {
writeContract({
address: TESSERACT_SWAP_ADDRESS,
abi: TESSERACT_SWAP_ABI,
functionName: 'swap',
args: [
tokenIn,
tokenOut,
parseEther(amountIn),
parseEther(amountOut),
address,
BigInt(Math.floor(Date.now() / 1000) + 3600),
],
});
};

return (
<div className="w-full max-w-md p-6 bg-white rounded-lg shadow-lg">
```

Swap

```
<div className="space-y-4">
  <div>
    <label className="block text-sm font-medium mb-2">From</label>
    <input
      type="number"
      value={amountIn}
      onChange={(e) => setAmountIn(e.target.value)}
      className="w-full p-2 border rounded"
      placeholder="0.0"
```

```
    />
    <select
      value={tokenIn}
      onChange={(e) => setTokenIn(e.target.value)}
      className="w-full p-2 border rounded mt-2"
    >
      <option value="">Select token</option>
      <option value="0x...">DOT</option>
      <option value="0x...">xcACA (Acala)</option>
    </select>
  </div>

  <div>
    <label className="block text-sm font-medium mb-2">To (estimated)</labe
    <input
      type="text"
      value={quote ? formatEther(quote) : '0.0'}
      disabled
      className="w-full p-2 border rounded bg-gray-50"
    />
    <select
      value={tokenOut}
      onChange={(e) => setTokenOut(e.target.value)}
      className="w-full p-2 border rounded mt-2"
    >
      <option value="">Select token</option>
      <option value="0x...">lstDOT</option>
      <option value="0x...">xcBNC (Bifrost)</option>
    </select>
  </div>

  <button
    onClick={handleSwap}
    disabled={!tokenIn || !tokenOut || !amountIn}
    className="w-full p-3 bg-blue-600 text-white rounded hover:bg-blue-700
  >
    Swap
  </button>
```

```
    </div>
  </div>
```

```
);
}
```

Staking Interface:

```
// components/StakePanel.tsx
'use client';

import { useState } from 'react';
import { useWriteContract, useReadContract, useAccount } from
'wagmi';
import { parseEther, formatEther } from 'viem';
import { TESSERACT_STAKE_ABI, TESSERACT_STAKE_ADDRESS }
from '@/lib/contracts';

export function StakePanel() {
const { address } = useAccount();
const [amount, setAmount] = useState('');
const { writeContract } = useWriteContract();

// Get exchange rate
const { data: exchangeRate } = useReadContract({
address: TESSERACT_STAKE_ADDRESS,
abi: TESSERACT_STAKE_ABI,
functionName: 'exchangeRate',
});

// Get user's lstDOT balance
const { data: lstDOTBalance } = useReadContract({
address: TESSERACT_STAKE_ADDRESS,
abi: TESSERACT_STAKE_ABI,
functionName: 'balanceOf',
args: [address],
});

const handleStake = () => {
writeContract({
address: TESSERACT_STAKE_ADDRESS,
```

abi: TESSERACT_STAKE_ABI,
functionName: 'stake',
args: [parseEther(amount)],
value: parseEther(amount),
});
};

return (

Liquid Staking

```
<div className="mb-4 p-4 bg-blue-50 rounded">
  <p className="text-sm text-gray-600">Current Exchange Rate</p>
  <p className="text-2xl font-bold">
    1 DOT = {exchangeRate ? formatEther(exchangeRate) : '...'} lstDOT
  </p>
</div>

<div className="mb-4 p-4 bg-green-50 rounded">
  <p className="text-sm text-gray-600">Your lstDOT Balance</p>
  <p class="text-xl font-semibold">
    {lstDOTBalance ? formatEther(lstDOTBalance) : '0.0'} lstDOT
  </p>
</div>

<div class="space-y-4">
  <div>
    <label class="block text-sm font-medium mb-2">Amount to Stake</l
    <input
      type="number"
      value={amount}
      onChange={(e) => setAmount(e.target.value)}
      class="w-full p-2 border rounded"
      placeholder="0.0 DOT"
    />
  </div>
```

```
      <button
        onClick={handleStake}
        disabled={!amount}
        className="w-full p-3 bg-green-600 text-white rounded hover:bg-green-7
      >
        Stake DOT
      </button>
    </div>

    <div className="mt-6 text-center text-sm text-gray-500">
      <p>Staking DOT gives you lstDOT tokens that earn rewards automatically<
    </div>
  </div>
```

);
}

Deployment

Build for Production:

npm run build
npm run start

Deploy to Vercel:

npm install -g vercel
vercel --prod

XCM Message Construction Guide

XCM Message Structure

XCM messages consist of a sequence of instructions executed on the destination chain. Messages must be SCALE-encoded before sending via the precompile[2].

Common XCM Patterns

Pattern 1: Withdraw + Buy Execution + Deposit

Used for simple cross-chain asset transfers:

```
WithdrawAsset (remove assets from holding register)
↓
BuyExecution (pay for execution on destination)
↓
DepositAsset (deposit to beneficiary account)
```

Pattern 2: Withdraw + Buy Execution + Transact

Used for executing smart contract calls on destination:

```
WithdrawAsset
↓
BuyExecution
↓
Transact (execute encoded call data)
```

XCM Encoding Helper Library

Create a Solidity library for common XCM patterns:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

library XCMHelpers {
// Encode simple transfer XCM message
function encodeTransferMessage(
uint256 amount,
bytes32 beneficiary
) internal pure returns (bytes memory) {
// Simplified - real implementation needs proper SCALE encoding
return abi.encodePacked(
hex"030c", // WithdrawAsset instruction
_encodeMultiAsset(amount),
hex"0401", // BuyExecution instruction
hex"04000000000000000000000000000000", // Execution weight
hex"0d01", // DepositAsset instruction
```

```
beneficiary // Beneficiary account
);
}
```

```
// Encode destination parachain location
function encodeParachainDestination(
    uint32 parachainId
) internal pure returns (bytes memory) {
    // V3 MultiLocation: { parents: 1, interior: X1(Parachain(id)) }
    return abi.encodePacked(
        hex"03",            // V3 version
        hex"01",            // parents = 1
        hex"00",            // X1 interior
        hex"00",            // Parachain junction
        parachainId         // Parachain ID (4 bytes)
    );
}

function _encodeMultiAsset(uint256 amount) private pure returns (bytes me
    // Concrete fungible asset encoding
    return abi.encodePacked(
        hex"00",            // Concrete asset
        hex"01",            // Interior: Here
        hex"01",            // Fungible
        amount              // Amount (encoded as compact)
    );
}
```

```
}
```

Using PAPI for XCM Construction

For complex XCM messages, use Polkadot API (PAPI) in TypeScript:

```
// scripts/generate-xcm-message.ts
import { createClient } from 'polkadot-api';
import { getWsProvider } from 'polkadot-api/ws-provider/web';
```

```
async function generateBifrostStakeXCM(amount: bigint) {
const client = createClient(
getWsProvider('wss://westend-hub-rpc.polkadot.io')
);

const xcmMessage = {
V3: [
{
WithdrawAsset: [
{
id: { Concrete: { parents: 0, interior: 'Here' } },
fun: { Fungible: amount }
}
]
},
{
BuyExecution: {
fees: {
id: { Concrete: { parents: 0, interior: 'Here' } },
fun: { Fungible: amount / 10n } // 10% for fees
},
weightLimit: 'Unlimited'
}
},
{
Transact: {
originKind: 'SovereignAccount',
requireWeightAtMost: { refTime: 1000000000n, proofSize: 64000n },
call: {
encoded: encodeBifrostMintCall(amount)
}
}
}
]
};

// Encode to SCALE format
const encoded = client.getUnsafeApi().tx.XcmPallet.send.encode({
dest: { V3: { parents: 1, interior: { X1: { Parachain: 2001 } } } },
```

```
message: xcmMessage
});

console.log('Encoded XCM message:', encoded);
return encoded;
}
```

Testing XCM Messages

Use Chopsticks for dry-run testing:

```
# Dry-run XCM message without executing

chopsticks dry-run
--endpoint wss://westend-hub-rpc.polkadot.io
--extrinsic 0x...
--at <block-hash>
```

Gas Optimization Strategies

Common Optimizations

1. **Pack storage variables:** Combine multiple small variables into single 32-byte slots
2. **Use calldata instead of memory:** For read-only function parameters
3. **Cache storage reads:** Read storage variables once, store in memory
4. **Short-circuit conditions:** Put cheaper checks first in require statements
5. **Batch operations:** Combine multiple actions to save on base transaction cost

Example - Storage Packing:

```
// ✗ Inefficient: Uses 3 storage slots
contract Bad {
uint128 public valueA; // Slot 0
```

```
uint128 public valueB; // Slot 1
uint256 public valueC; // Slot 2
}

// ✅ Optimized: Uses 2 storage slots
contract Good {
uint128 public valueA; // Slot 0 (first 16 bytes)
uint128 public valueB; // Slot 0 (last 16 bytes)
uint256 public valueC; // Slot 1
}
```

Example - Caching Storage Reads:

```
// ❌ Inefficient: Reads storage 3 times
function calculateBad() external view returns (uint256) {
return totalStaked + totalStaked * rewardRate / 100;
// 'totalStaked' read twice from storage
}

// ✅ Optimized: Reads storage once
function calculateGood() external view returns (uint256) {
uint256 cached = totalStaked; // Read once
return cached + cached * rewardRate / 100;
}
```

Foundry Gas Profiling

Enable gas reporting in tests:

```
// Add to test file
function testSwapGasUsage() public {
uint256 gasBefore = gasleft();
```

```
swap.swap(
    address(tokenA),
    address(tokenB),
    10 ether,
    1 ether,
    alice,
    block.timestamp + 3600
```

```
);

uint256 gasUsed = gasBefore - gasleft();
console.log("Gas used for swap:", gasUsed);
```

}

Run with gas reporting:

forge test --gas-report

Security Considerations

Critical Security Checks

- **Reentrancy:** Use OpenZeppelin's ReentrancyGuard on all state-changing functions
- **Integer overflow:** Solidity 0.8.x has built-in checks; for unchecked math, verify manually
- **Access control:** Use Ownable or AccessControl for admin functions
- **Slippage protection:** Always require minimum output amounts in swaps
- **Deadline checks:** Validate transaction deadlines to prevent stale transactions
- **XCM message validation:** Verify XCM messages are properly formatted and signed

Common Vulnerabilities to Avoid

Reentrancy Attack:

```
// ✗ Vulnerable
function withdraw(uint256 amount) external {
require(balances[msg.sender] >= amount);
(bool success,) = msg.sender.call{value: amount}(""); // External call
before state update
require(success);
balances[msg.sender] -= amount; // State updated AFTER external call
}
```

```
// ✅ Protected
function withdraw(uint256 amount) external nonReentrant {
require(balances[msg.sender] >= amount);
balances[msg.sender] -= amount; // State updated BEFORE external
call
(bool success,) = msg.sender.call{value: amount}("");
require(success);
}
```

Front-running Protection:

```
function swap(
address tokenIn,
address tokenOut,
uint256 amountIn,
uint256 minAmountOut, // Minimum acceptable output
address to,
uint256 deadline // Transaction must execute before deadline
) external {
require(block.timestamp <= deadline, "Expired");

    uint256 amountOut = _calculateAmountOut(tokenIn, tokenOut, amountIn);
    require(amountOut >= minAmountOut, "Slippage too high");

    // Execute swap...

}
```

Testing Security

Run Slither static analyzer:

```
pip install slither-analyzer
slither src/TesseractSwap.sol --exclude-dependencies
```

Common issues Slither detects:

- Reentrancy vulnerabilities
- Unprotected state variables
- Missing events on important state changes

- Incorrect access control

Deployment Checklist

Pre-Deployment

```
All tests passing (unit + integration)
Gas optimization completed
Slither security scan passed
Contract verified on block explorer (if available)
Documentation complete (README, inline comments)
Frontend connected to testnet contracts
Demo video recorded
```

Deployment Steps

1. Deploy to Polkadot Hub Testnet:

```
# Load environment variables
source .env

# Deploy TesseractSwap
forge script script/DeploySwap.s.sol
--rpc-url $POLKADOT_HUB_RPC
--private-key $PRIVATE_KEY
--broadcast
--verify

# Deploy TesseractStake
forge script script/DeployStake.s.sol
--rpc-url $POLKADOT_HUB_RPC
--private-key $PRIVATE_KEY
--broadcast
```

Deploy TesseractVault

```
forge script script/DeployVault.s.sol
--rpc-url $POLKADOT_HUB_RPC
--private-key $PRIVATE_KEY
--broadcast
```

2. Verify Contracts:

```
forge verify-contract
--chain-id <polkadot-hub-chain-id>
--compiler-version 0.8.20
<contract-address>
src/TesseractSwap.sol:TesseractSwap
```

3. Update Frontend with Contract Addresses:

```
// lib/contracts.ts
export const TESSERACT_SWAP_ADDRESS = '0x...' as const;
export const TESSERACT_STAKE_ADDRESS = '0x...' as const;
export const TESSERACT_VAULT_ADDRESS = '0x...' as const;
```

4. Test Live Deployment:

- Connect frontend to testnet
- Execute test swap
- Execute test stake
- Execute test vault deposit
- Verify all transactions on block explorer

Post-Deployment

Set Up On-Chain Identity:

Required for winning teams per hackathon rules[9]:

1. Visit Polkadot.js Apps: https://polkadot.js.org/apps/
2. Connect wallet with team DOT tokens
3. Navigate to Accounts → Set on-chain identity
4. Fill in: Display name, email, website, GitHub
5. Submit transaction (costs ~20 DOT deposit)

GitHub Repository Setup:

Repository is public
README.md with clear installation instructions
LICENSE file (MIT recommended)
All dependencies documented
.gitignore properly configured
Commit history shows hackathon contributions

Documentation Requirements

README.md Structure

Tesseract - Cross-Chain DeFi Primitives on Polkadot

Overview

Brief description of what Tesseract does and why it's unique.

Architecture

- TesseractSwap: XC-20 AMM DEX
- TesseractStake: Native liquid staking
- TesseractVault: Cross-chain yield optimizer

Features

- Cross-chain asset swaps via XCM
- Native DOT liquid staking with lstDOT
- Automated yield routing across parachains

Tech Stack

- Solidity 0.8.20
- Foundry
- Next.js 14

- Wagmi v2

Installation

Prerequisites

- Node.js 18+
- Foundry

Setup

```
git clone https://github.com/your-team/tesseract-defi
cd tesseract-defi

# Install dependencies
forge install
cd frontend && npm install

# Set up environment
cp .env.example .env
# Edit .env with your keys
```

Testing

```
forge test -vvv
```

Deployment

```
forge script script/Deploy.s.sol --rpc-url $POLKADOT_HUB_RPC --broadcast
```

Live Demo

- Frontend: https://tesseract-demo.vercel.app
- Contracts: [Link to Subscan explorer]

Demo Video

[Link to 3-5 minute demo video]

Team

- Developer 1: [GitHub]
- Developer 2: [GitHub]

Hackathon Tracks

- Track 1: EVM Smart Contracts (TesseractSwap)
- Track 2: PVM Smart Contracts (TesseractStake)

License

MIT

Inline Documentation

Add NatSpec comments to all public/external functions:

```
/// @notice Swaps exact input tokens for output tokens
/// @dev Implements constant product AMM formula with 0.3% fee
/// @param tokenIn Address of input token
/// @param tokenOut Address of output token
/// @param amountIn Amount of input tokens
/// @param minAmountOut Minimum acceptable output (slippage
protection)
/// @param to Recipient address
/// @param deadline Transaction must execute before this timestamp
/// @return amountOut Actual amount of output tokens received
function swap(
address tokenIn,
address tokenOut,
```

```
uint256 amountIn,
uint256 minAmountOut,
address to,
uint256 deadline
) external returns (uint256 amountOut) {
// Implementation...
}
```

Demo Video Script

Duration: 3-5 minutes

Structure:

1. **Introduction (30 seconds)**
    - Team name and hackathon tracks
    - Tesseract overview: Cross-chain DeFi primitives
    - Why Polkadot Hub enables unique functionality
2. **TesseractSwap Demo (1 minute)**
    - Show frontend swap interface
    - Select DOT → xcACA (Acala) pair
    - Execute swap
    - Highlight XCM cross-chain transfer
    - Show transaction on Subscan
3. **TesseractStake Demo (1 minute)**
    - Show staking interface
    - Stake DOT, receive lstDOT
    - Explain exchange rate mechanism
    - Show lstDOT balance increasing with rewards
4. **TesseractVault Demo (1.5 minutes)**
    - Show vault dashboard with APY comparison
    - Deposit lstDOT into vault
    - Explain multi-strategy allocation
    - Show XCM routing to Bifrost and Acala
    - Display projected returns
5. **Architecture Overview (1 minute)**
    - Show architecture diagram
    - Explain XCM precompile integration
    - Highlight PVM usage for staking

    - Mention open-source composability
6. **Closing (30 seconds)**
    - Recap value proposition
    - Future roadmap (W3F grant application)
    - Call to action: GitHub repo, live demo link

Recording Tips:

- Use screen recording software (OBS, Loom)
- Prepare script and practice beforehand
- Show actual working demo, not slides
- Include voiceover explaining each step
- Add text overlays for key points
- Export in high quality (1080p minimum)

Hackathon Submission Checklist

Bare Minimum Requirements

All code is open-source (MIT license)
Public GitHub repository
Codebase <70% similar to any existing project
All team members verified identity on Polkadot Official Discord
Valid commit history during hackathon period (Feb 15 - Mar 24)
Only hackathon-period code considered for scoring

Winner Criteria

Polkadot on-chain identity set up for team wallet
Quality documentation (README, GitBook, or website)
Good UI/UX (frontend is usable and intuitive)
Demo video or screenshots included
Hosted deployment OR local installation guide
Clear roadmap and commitment to future development
Solution is valid for submitted track

Track Relevance

Track 1 (EVM Smart Contracts):

> TesseractSwap deployed on Polkadot Hub
> Uses Solidity and EVM-compatible tools
> Qualifies as DeFi-enabled dapp

Track 2 (PVM Smart Contracts):

> TesseractStake uses Polkadot native assets (DOT)
> Accesses native functionality via precompiles (staking)
> Optional: Demonstrates calling Rust/C++ from Solidity

Submission Package

What to submit (by March 20, 2026):

1. GitHub repository URL
2. Live demo URL (Vercel deployment)
3. Demo video URL (YouTube, Loom, or uploaded)
4. Documentation link (README or GitBook)
5. Team member Polkadot addresses (for identity verification)
6. Brief project description (100-200 words)

Where to submit:

Check hackathon platform for submission portal. Likely:

- OpenGuild hackathon website
- Dorahacks or similar platform
- Submit via form or email to zoey@openguild.wtf

Troubleshooting Guide

Common Issues and Solutions

Issue: Contract deployment fails with "insufficient gas"

Solution:

Increase gas limit in deployment script

```
forge script script/Deploy.s.sol
--rpc-url $RPC_URL
--gas-limit 5000000
--broadcast
```

Issue: XCM precompile calls revert

Possible causes:

- Incorrect SCALE encoding of XCM message
- Invalid destination parachain ID
- Insufficient balance for execution fees

Solution:

- Test XCM encoding with Chopsticks first
- Verify parachain ID in Polkadot.js Apps
- Check contract has enough DOT/WND for fees

Issue: Staking precompile not found

If staking precompile is not yet available:

1. Check official docs: https://docs.polkadot.com/smart-contracts/precompiles/
2. Ask in OpenGuild Discord: https://discord.gg/WWgzkDfPQF
3. Implement alternative: XCM-based liquid staking via Bifrost

Issue: Frontend can't connect to MetaMask

Solution:

- Verify Polkadot Hub network is added to MetaMask
- Check RPC URL is correct and accessible
- Ensure chain ID matches deployment network

Issue: Tests fail with "EVM revert"

Solution:

Run with increased verbosity to see revert reason

forge test -vvvv

Enable detailed traces

forge test --trace

Getting Help

Resources:

- **Polkadot Developer Support:** https://t.me/substratedevs
- **OpenGuild Discord:** https://discord.gg/WWgzkDfPQF
- **Hackathon Contact:** Telegram @Zoey1412
- **Polkadot Docs:** https://docs.polkadot.com/
- **Stack Exchange:** https://substrate.stackexchange.com/

Post-Hackathon Roadmap

Immediate Next Steps (Week 1-2)

1. Incorporate feedback from judges and demos
2. Fix any critical bugs discovered during hackathon
3. Polish documentation and UI
4. Prepare W3F grant application

Short-Term Goals (Month 1-3)

1. **Audit and Security:**
    - Professional smart contract audit (OpenZeppelin, Peckshield)
    - Bug bounty program launch
2. **Feature Expansion:**
    - Additional strategy integrations (Moonbeam, Astar)

    - Advanced swap features (multi-hop routing, limit orders)
    - Governance token launch for TesseractDAO
3. **Mainnet Deployment:**
    - Deploy to Polkadot Hub mainnet
    - Liquidity bootstrapping event
    - Partner with existing Polkadot protocols

Long-Term Vision (Month 3-12)

1. **Ecosystem Integration:**
    - Integration with Hyperbridge for Ethereum ↔ Polkadot flows
    - Support for more parachains (Interlay, HydraDX, Phala)
    - Cross-ecosystem bridges (Cosmos, Avalanche)
2. **Advanced Features:**
    - AI-powered yield optimization using off-chain oracles
    - Automated strategy rebalancing based on market conditions
    - Risk management and insurance integration
3. **Decentralization:**
    - DAO governance for strategy selection
    - Community-submitted strategies
    - Progressive decentralization of admin controls

W3F Grant Application

Grant Track: Core infrastructure

Milestones:

Milestone	Deliverable	Duration
M1	Security audit + bug fixes	1 month
M2	Additional strategy integrations	2 months
M3	Mainnet deployment + documentation	1 month
M4	DAO governance implementation	2 months

Table 3: Proposed W3F grant milestones

Requested Amount: $50,000 - $100,000

Justification: Tesseract provides critical DeFi infrastructure for Polkadot Hub, enabling cross-chain composability that benefits the entire ecosystem.

Conclusion

This technical specification provides a complete, actionable guide to building Tesseract for the Polkadot Solidity Hackathon. The architecture is designed to:

1. Fully satisfy both Track 1 (EVM) and Track 2 (PVM) requirements
2. Demonstrate Polkadot-native functionality impossible on Ethereum
3. Provide real utility as foundational DeFi infrastructure
4. Meet all hackathon judging criteria
5. Position for W3F grant and long-term success

By following this guide, any AI agent or developer team can build Tesseract from scratch, deploy it successfully, and compete for top prizes. The modular architecture allows work to proceed in parallel across three independent modules, enabling efficient team collaboration.

Key Success Factors:

- Start with TesseractSwap (simplest, proves XCM integration)
- Monitor for staking precompile availability; have Bifrost backup plan
- Prioritize working demo over perfect code
- Document extensively as you build
- Test XCM messages thoroughly with Chopsticks
- Record demo video early; iterate if needed
- Submit before deadline with buffer time

The Tesseract name represents exactly what this system does: connects dimensions (chains) that normally cannot interact, creating a unified cross-chain DeFi experience unique to Polkadot.

Good luck building!

References

[1] Parity Technologies. (2026). Polkadot Hub: Smart Contracts on Polkadot. *Polkadot Support*. https://support.polkadot.network/support/solutions/articles/65000191304

[2] Polkadot Developer Documentation. (2026). Interact with the XCM Precompile. *Polkadot Docs*. https://docs.polkadot.com/smart-contracts/precompiles/xcm/

[3] Moonbeam Network. (2022). Introducing XC-20s: The New Standard for Cross-Chain Tokens. *Moonbeam News*. https://moonbeam.network/news/introducing-xc-20s-the-new-standard-for-cross-chain-tokens-on-dotsama/

[4] Lido Finance. (2023). Liquid Staking on Polkadot. *Lido DOT Documentation*. https://docs.polkadot.lido.fi/fundamentals/liquid-staking/

[5] Polkadot Developer Documentation. (2026). Interact with the System Precompile. *Polkadot Docs*. https://docs.polkadot.com/smart-contracts/precompiles/system/

[6] Bifrost Finance. (2025). Bifrost: The Polkadot Liquid Staking Rollup. *Polkadot Support*. https://support.polkadot.network/support/solutions/articles/65000190313

[7] Frontiers in Blockchain. (2024). Protocol for unifying cross-chain liquidity on Polkadot. *Frontiers Journal*, 7. https://www.frontiersin.org/journals/blockchain/articles/10.3389/fbloc.2024.1413840/full

[8] Polkadot Foundation. (2026). Westend Faucet. https://faucet.polkadot.io/westend

[9] OpenGuild. (2026). Polkadot Solidity Hackathon: Judgment Criteria. *Hackathon Documentation*. https://openguild.wtf/blog/polkadot/polkadot-opengov-introduction